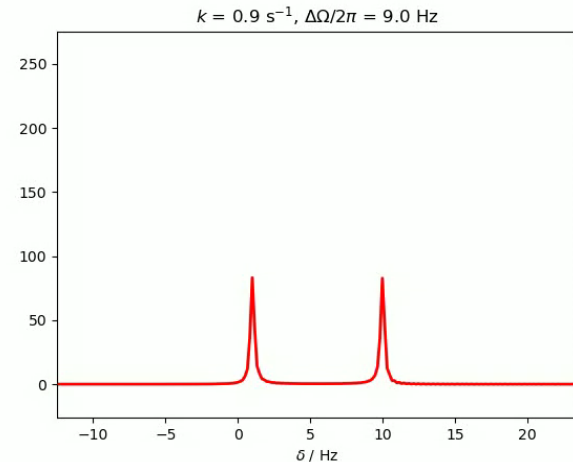


SLEEPY Tutorial

Simulating relaxation and exchange

Andy Smith

Ampere NMR School 2026



Simulation requirements 1: Hilbert space (coherent)

1. A density matrix:

- Describes the state of the spin system
- Build out of spin matrices
- $n \times n$ dimensional matrix, for system with n eigenstates

Spin-1/2 spin matrices

$$\hat{S}_x = \frac{1}{2} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \hat{S}_y = \frac{i}{2} \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

$$\hat{S}_z = \frac{1}{2} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad \hat{1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Simulation requirements 1: Hilbert space (coherent)

1. A density matrix
2. Hamiltonians:
 - Spin-field interactions (e.g. Zeeman, chemical shift, rf-fields)
 - Spin-spin interaction (e.g. J-coupling, dipole, quadrupole)

$$\hat{H}^{CS,n} = 2\pi\delta_{CS,i}\hat{S}_z^{(n)} = 2\pi\delta_{CS,n}\frac{1}{2}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

$$\hat{H}^{J,(m,n)} = 2\pi J\hat{S}_z^{(m)} \cdot \hat{S}_z^{(n)} = 2\pi J\frac{1}{4}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Spin matrices: Multi-spin systems

- Use the Kronecker product to ($\otimes$) expand the spin system

e.g. 2 spin system

$$\hat{S}_z^{(1)} = \hat{S}_z \otimes \hat{1} = \frac{1}{2} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \quad \hat{S}_z^{(2)} = \hat{1} \otimes \hat{S}_z = \frac{1}{2} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

$$\hat{S}_x^{(1)} = \hat{S}_x \otimes \hat{1} = \frac{1}{2} \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \quad \hat{S}_y^{(2)} = \hat{1} \otimes \hat{S}_y = \frac{i}{2} \begin{pmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

More spins:

$$\hat{S}_x^{(1)} = \hat{S}_x \otimes \hat{1} \otimes \hat{1} \quad \hat{S}_z^{(2)} = \hat{1} \otimes \hat{S}_z \otimes \hat{1} \quad \hat{S}_y^{(3)} = \hat{1} \otimes \hat{1} \otimes \hat{S}_y$$

How to do it in SLEEPY

```
op=sl.SpinOp([1/2,1/2,1/2])  
op[2].y
```

$$\begin{aligned} \hat{S}_x &= \frac{1}{2} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\ \hat{S}_y &= \frac{i}{2} \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \\ \hat{S}_z &= \frac{1}{2} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \\ \hat{1} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \end{aligned}$$

Simulation requirements 1: Hilbert space (coherent)

1. A density matrix
2. Hamiltonians
3. Detection matrix
 - Defines what we want to measure
 - 1 spin coherence, multi-spin coherences, polarization, etc.
 - Construct out of spin matrices
 - (take the complex transpose of the desired operator)

e.g. coherence on spin 1 $\hat{O}_{detect} = \left(\hat{S}_+^{(1)} \right)^\dagger = \left(\hat{S}_x^{(1)} + i\hat{S}_y^{(1)} \right)^\dagger$

Put the pieces together

1. Propagate density matrix under the Hamiltonian

$$\hat{\rho}(t + \Delta t) = \exp(-i\hat{H}\Delta t) \hat{\rho}(t) \exp(i\hat{H}\Delta t)$$

2. Detect the signal

$$\text{signal}(t) = \text{trace} \left(\hat{O}_{\text{detect}} \cdot \hat{\rho}(t) \right)$$

Trace is the diagonal

3. Repeat as needed

Further steps

Time dependent Hamiltonian, e.g. due to MAS
Orientation dependence / powder averaging
Faster calculation, e.g. use eigenbasis,
etc.

Major limitation: No dissipative processes

No relaxation
No stochastic changes in Hamiltonian,
i.e. dynamics

Simulation requirements 2: Liouville space

Requirements

1. A density matrix:
 - stretch matrices into column
2. Hamiltonians:
 - Convert into the Liouvillian
 - $N \times N$ becomes $N^2 \times N^2$
$$\hat{L}_{coh} = -i * (\hat{H} \otimes \hat{1} - \hat{1} \otimes \hat{H}')$$
3. Detection matrix :
 - stretch matrices into column

Steps

1. Propagate

$$\hat{\rho}(t + \Delta t) = \exp(\hat{L}_{coh} \Delta t) \cdot \hat{\rho}(t)$$

2. Detect

$$signal(t) = \hat{O}_{detect} \cdot \hat{\rho}(t)$$

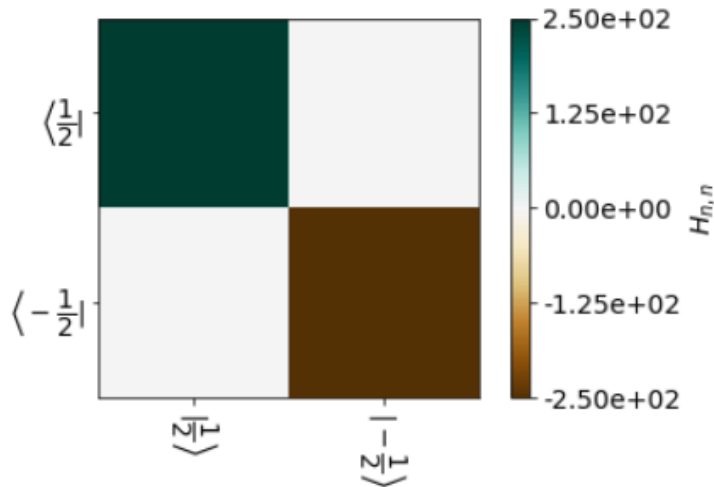
3. Repeat as need

Liouville Space: Exchange

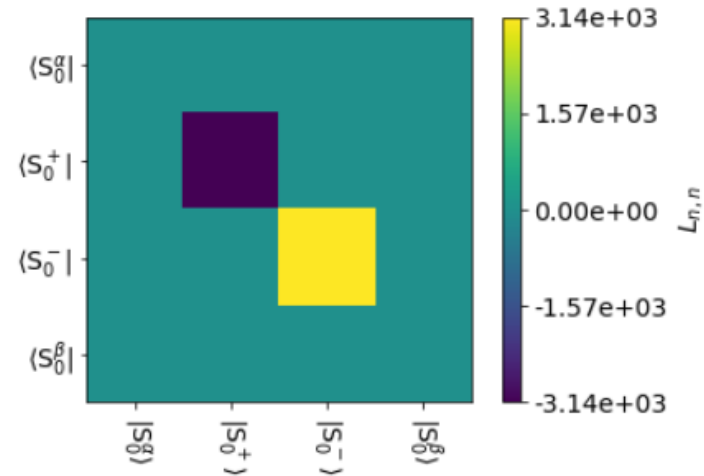
- So far, this will just make our simulations slow
- But what if we have *dynamics*
 - Simulate effects via exchange in Liouville space!

Liouville Space: Exchange

1. Build Hamiltonians, convert to Liouville space



Hamiltonian (Hilbert space)



Liouvillian for 1 Hamiltonian

SLEEPY

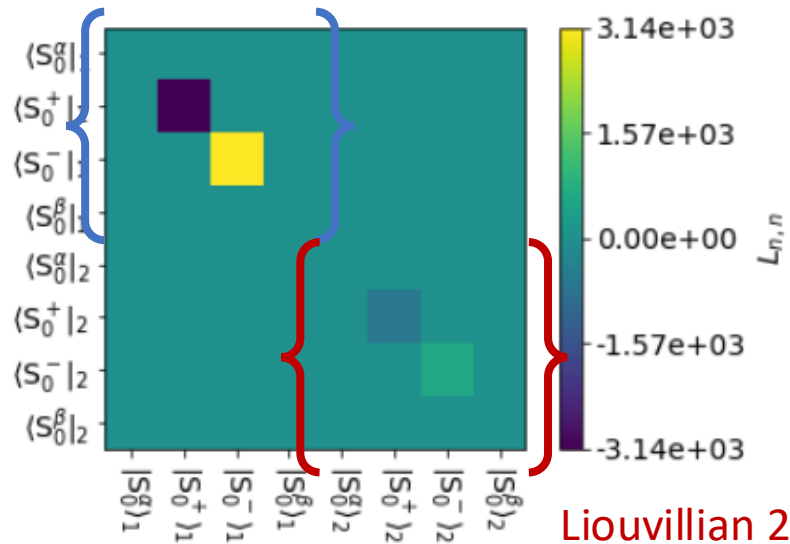
```
H=ex.Hamiltonian()  
H.plot(mode='re')
```

```
L=ex.Liouvillian()  
_=L.plot(mode='im')
```

Liouville Space: Exchange

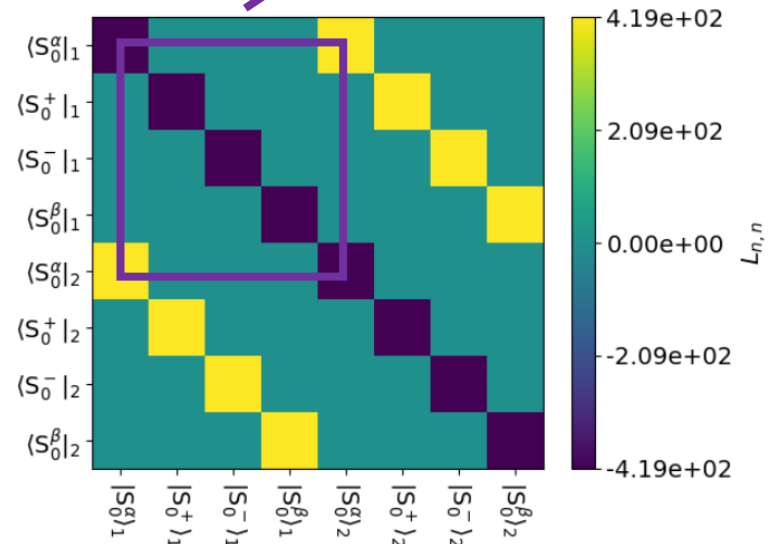
2. Couple multiple Liouvillians with an exchange matrix

Liouvillian 1 (imaginary terms)



Liouvillian 2

+



One copy of mass-conserving
(columns sum to zero)
exchange matrix

Here, we have 4 copies total

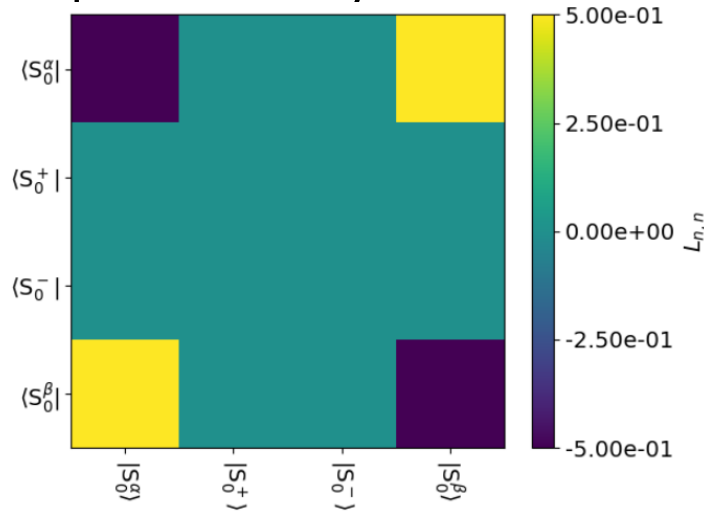
SLEEPY

```
L.plot('Lcoh', mode='im')
```

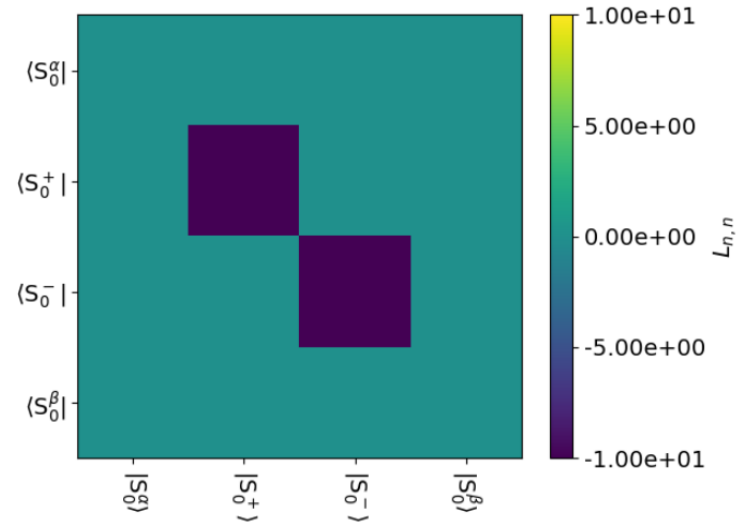
```
L.plot('Lex', mode='re')
```

Liouville Space: Relaxation

- Couple or destroy states



T_1 relaxation: couple $\hat{S}^\alpha$
and $\hat{S}^\beta$ states with exchange matrix



T_2 relaxation: Destroy coherences with
real, negative terms

SLEEPY

```
ex=sl.ExpSys(600,Nucs='1H')
L=ex.Liouvillian().add_relax('T1',i=0,T1=1)
L.plot('Lrelax',mode='re')
```

```
ex=sl.ExpSys(600,Nucs='1H')
L=ex.Liouvillian().add_relax('T2',i=0,T2=.1)
L.plot('Lrelax',mode='re')
```

SLEEPY Cookbook

1. Define experimental conditions

```
ex0=sl.ExpSys(500,Nucs=['15N','1H'],vr=60000)
```

2. Add Hamiltonians

```
ex0.set_inter('dipole',i0=0,i1=1,delta=22000)  
ex0.set_inter('CSA',i=0,delta=113)
```

3. For exchange, copy and modify

```
ex1=ex0.copy()  
ex1.set_inter('dipole',i0=0,i1=1,delta=delta,euler_d=[0,20,0])  
ex1.set_inter('CSA',i=0,delta=113,euler_d=[0,20,0])
```

4. Create Liouvillian

```
L=sl.Liouvillian(ex0,ex1,kex=sl.Tools.twoSite_kex(tc=1e-4))
```

5. Add relaxation

```
L.add_relax('SpinDiffusion',i=1,k=20)
```

6. Define pulse sequence

```
seq=L.Sequence()  
seq.add_channel('15N',v1=15000)
```

7. Define initial density matrix/detection

```
rho=sl.Rho(rho0='15Nx',detect='15Nx')
```

8. Propagate

```
rho.DetProp(seq,n=12000)
```

Lets try it!